

실전 웹서비스개발 - 2주차

웹 개발 핵심 개념 이해와 웹 프레임워크를 활용한 개발 및 배포

학습 목표

- 웹 핵심 구성 요소
- 웹 사이트 렌더링 방식
- 웹 개발 프레임워크란?
 - SvelteKit
- 클라우드 서비스 배포
 - Vercel

웹 개발 핵심 개념 이해

웹 문서의 기본 구조 이해

HTML = 구조, CSS = 표현(스타일), JS = 동작

<!DOCTYPE html>

- 현재 문서가 HTML5 문서임을 브라우저에 알려주는 선언문

<html> 태그

- 웹 문서의 시작과 끝을 감싸는 최상위 루트 요소
- 모든 HTML 요소는 이 안에 포함

<head> 태그

- HTML 문서의 메타데이터(제목, 인코딩, 스타일, 스크립트 등)를 담는 영역

<meta> 태그

- 문서의 부가 정보를 정의하는 메타데이터 요소
- 웹 페이지 메타데이터
- viewport: 반응형 디자인의 핵심
- link 태그: 외부 리소스 연결

<body> 태그

- 사용자가 실제로 화면에서 보게 되는 모든 콘텐츠(텍스트, 이미지, 링크 등)를 담는 영역

HTML5 문서 구조

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>실전웹서비스개발 2주차</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>환영합니다 !</h1>
    <p>이것은 나의 첫 번째 웹사이트입니다 </p>
  </header>

  <main>
    <section>
      <h2>자기소개</h2>
      <p id="intro">안녕하세요. 저는 웹 개발을 공부하고 있습니다.</p>
      <button onclick="changeText()">클릭해보세요</button>
    </section>
  </main>

  <footer>
    <p>&copy; 2025 PWD Week 2</p>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

학습 참고 자료 : <https://www.w3schools.com/html/default.asp>

웹 문서의 기본 구조 이해

HTML = 구조, CSS = 표현(스타일), JS = 동작

규칙(Selector + Declaration)의 집합

Selector

- 스타일을 적용할 대상을 가리키는 부분
예) h1, .btn, #app, a:hover, input[type="text"]
- 태그/클래스/ID: h1, .btn, #app
- 관계자: 자식(>), 자손(공백), 형제(+, ~)
- 상태/의사클래스: :hover, :focus-visible, :disabled
- 의사요소: ::before, ::after

Declaration

- 중괄호 {} 안의 속성: 값; 한 쌍.
예) color: #111; margin-top: 8px;
- 여러 개 선언이 모이면 declaration block

HTML5 문서 구조

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
  color: #333;
}

main {
  max-width: 800px;
  margin: 2rem auto;
  padding: 0 1rem;
}

section {
  background: #f4f4f4;
  padding: 2rem;
  border-radius: 8px;
  margin-bottom: 2rem;
}

button:hover {
  background: #45a049;
}
```

학습 참고 자료 : <https://www.w3schools.com/css/default.asp>

웹 문서의 기본 구조 이해

HTML = 구조, CSS = 표현(스타일), JS = 동작

사용자 상호작용과 데이터를 이용해 동작을 구현

핵심 흐름

1. 브라우저가 HTML을 파싱 → DOM 트리 생성
2. CSS 파싱 → CSSOM 생성 → 렌더 트리 구성
3. JS가 DOM/CSSOM을 읽고 수정
4. 화면 업데이트 파이프라인
 - a. 스타일 재계산
 - b. 레이아웃
 - c. 페인트
 - d. 컴포지트

index.html

```
<body>
  ...
  <button onclick="changeText()">클릭해보세요</button>
  <script src="script.js" defer></script>
  ...
</body>
```

script.js

```
function changeText() {
  const intro = document.getElementById('intro');
  const messages = [
    '웹 개발은 정말 재미있어요!',
    'JavaScript로 상호작용을 만들 수 있습니다.',
    'GitHub Pages로 무료 호스팅이 가능해요!',
    '계속 공부하면 더 멋진 것을 만들 수 있어요!'
  ];

  const randomMessage = messages[Math.floor(Math.random() *
messages.length)];
  intro.textContent = randomMessage;
}

// 페이지 로드 시 환영 메시지
window.addEventListener('load', () => {
  console.log('웹사이트가 성공적으로 로드되었습니다!');
});
```

학습 참고 자료 : <https://www.w3schools.com/js/default.asp>

DOM

Document Object Model

- HTML 문서를 파싱 후메모리 안의 객체 트리로 바꾼 것
- JS로 읽고/바꾸는 대상
- 트리 구조(부모/자식/형제)
- HTML 파싱하면서 DOM 트리가 만들어짐

구성

- Node(모든 것의 부모 개념)
- Element(태그)
- Text(텍스트)
- Attribute(속성)

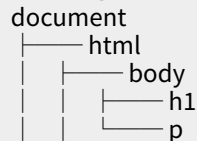
주요 작업

- 찾기
- 읽기/쓰기
- 생성/삭제

html

```
<html>
  <body>
    <h1>제목</h1>
    <p>내용</p>
  </body>
</html>
```

HTML 텍스트 → 파싱 → DOM 트리 (객체)



js

```
// 요소 찾기
const title = document.querySelector('h1');

// 내용 변경
title.textContent = '새로운 제목';

// 스타일 변경
title.style.color = 'red';

// 새 요소 추가
const newP = document.createElement('p');
newP.textContent = '추가된 단락';
document.body.appendChild(newP);
```

학습 참고 자료 : https://www.w3schools.com/jsref/dom_obj_document.asp

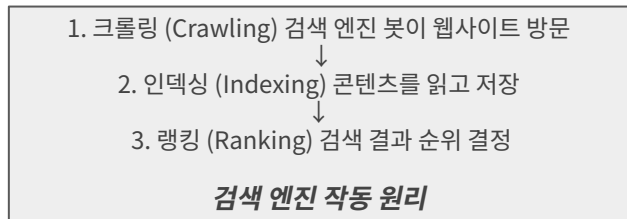
시맨틱 HTML

시맨틱 HTML의 중요성

- SEO 최적화

SEO(Search Engine Optimization)?

웹페이지의 태그와 링크 구조를 최적화하여 페이지가 검색엔진의 검색 결과 페이지에서 잘 노출되도록 최적화하는 과정



- 접근성 트리에 올바른 역할(role)로 매핑

접근성 트리 (Accessibility Tree)

브라우저가 DOM(+CSS, ARIA)을 해석해 만든 보조기기용 안내도

- 코드 가독성 개선
- 탄탄한 기본 UX & 내구성

✗ Non-Semantic

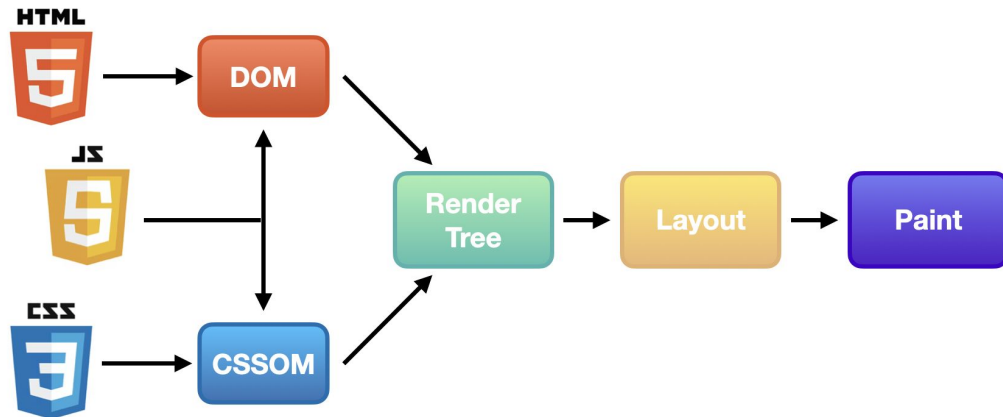
```
<div class="header">
  <div class="nav">...</div>
</div>
<div class="content">
  <div class="article">...</div>
  <div class="sidebar">...</div>
</div>
<div class="footer">...</div>
```

✓ Semantic

```
<header>
  <nav>...</nav>
</header>
<main>
  <article>...</article>
  <aside>...</aside>
</main>
<footer>...</footer>
```

웹 브라우저 렌더링 과정

1. HTML 파싱 → DOM 트리
2. CSS 파싱 → CSSOM 트리
3. DOM + CSSOM → Render Tree
4. Layout (위치, 크기 계산)
5. Paint (픽셀로 그리기)



웹 브라우저 렌더링 방식

CSR (Client-Side Rendering)

- 빈 HTML → JS 다운로드 → 화면 생성
- 장점: 페이지 로드 후 사용자 인터랙션에 빠르게 반응, 자주 업데이트되는 페이지에 유리
- 단점: 초기 로딩 느림, 구글 크롤러가 자바스크립트를 실행해야만 페이지 내용을 인식하므로 SEO에 불리
- 사례: 소셜 미디어 플랫폼(Facebook, Instagram), 대시보드, 웹 기반 게임

SSR (Server-Side Rendering)

- 서버에서 완성된 HTML → 즉시 표시
- 장점: 빠른 초기 로딩, SEO에 유리(검색엔진이 쉽게 페이지를 크롤링)
- 단점: 서버 부하(서버 비용이 증가)
- 사례: 뉴스 사이트 및 블로그, 전자 상거래 사이트(SEO가 중요)

SSG (Static Site Generation)

- 빌드 시 HTML 생성 → CDN 제공
- 장점: 빠른 로딩, SEO에 유리, 서버 부하 적음, 전세계적으로 빠르게 콘텐츠 제공 가능
- 단점: 동적인 콘텐츠가 많은 경우 콘텐츠 빌드 배포하는 과정이 필요.
- 사례: 개인 블로그 또는 포트폴리오 사이트, 마케팅 랜딩 페이지

웹 브라우저 렌더링 성능의 중요성

렌더링 최적화는 단순히 속도만의 문제가 아니라 사용자 경험, SEO, 전환율에 직접적인 영향을 미치는 핵심 요소

성능 지표와 사용자 경험

- 로딩 시간별 이탈률:

- 1초: 7% 이탈
- 3초: 32% 이탈
- 5초: 90% 이탈
- 7초: 103% 이탈 (경쟁사로 이동)

- Google Core Web Vitals

- Google이 2020년에 도입한 웹 페이지 사용자 경험을 측정하는 3가지 핵심 지표.
- 2021년부터는 검색 순위에도 영향을 미치는 중요한 SEO 요소
- FCP (First Contentful Paint): 첫 번째 텍스트나 이미지가 화면에 그려지는 시간
- LCP (Largest Contentful Paint): 가장 큰 콘텐츠가 화면에 그려지는 시간
- CLS (Cumulative Layout Shift): 페이지 로드 중 발생하는 레이아웃 이동 총합



Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

▲ 0-49 ■ 50-89 ● 90-100

라우팅(Routing)

사용자가 요청한 URL 주소에 따라 적절한 페이지나 콘텐츠를 보여주는 과정

<https://mysite.com/about>

↓
라우터가 확인

↓
"about 페이지를 보여줘!"

↓
About 페이지 표시

Routing 개념

URL 구조와 라우팅

<https://example.com/blog/2024/first-post?sort=date#comments>

해시 (페이지 내 위치)
쿼리 파라미터
동적 세그먼트
경로 세그먼트
경로 (path)
도메인
프로토콜

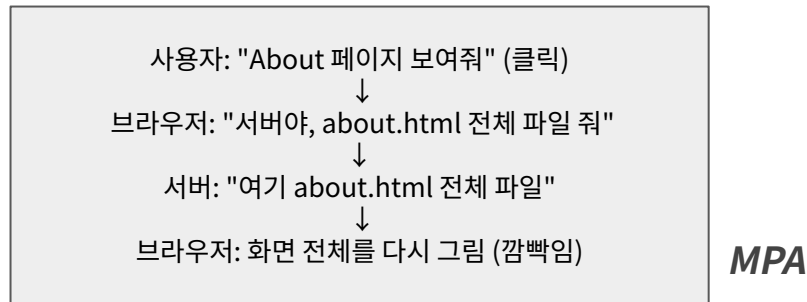
라우팅(Routing)

1. 서버 사이드 라우팅 (MPA 방식)

매번 새로운 페이지를 서버에서 받아옴

특징:

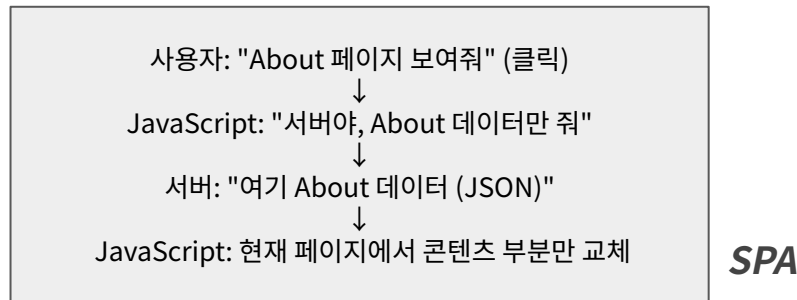
- ☒ 간단하고 직관적
- ☒ SEO에 유리
- ☒ 페이지 전환마다 깜빡임
- ☒ 느린 페이지 전환



2. 클라이언트 사이드 라우팅 (SPA 방식)

JavaScript가 페이지 전환을 처리

- ☒ 빠른 페이지 전환
- ☒ 부드러운 사용자 경험
- ☒ 초기 로딩이 느림
- ☒ SEO 대응 필요



SPA (Single Page Application) : 단일 페이지 애플리케이션

웹사이트 전체가 하나의 페이지로 구성되어 있고, 사용자가 다른 페이지로 이동할 때 전체 페이지를 새로 불러오지 않고 필요한 부분만 JavaScript로 동적으로 변경

웹 브라우저에서 확인 (크롬 브라우저 기준)

개발자 도구

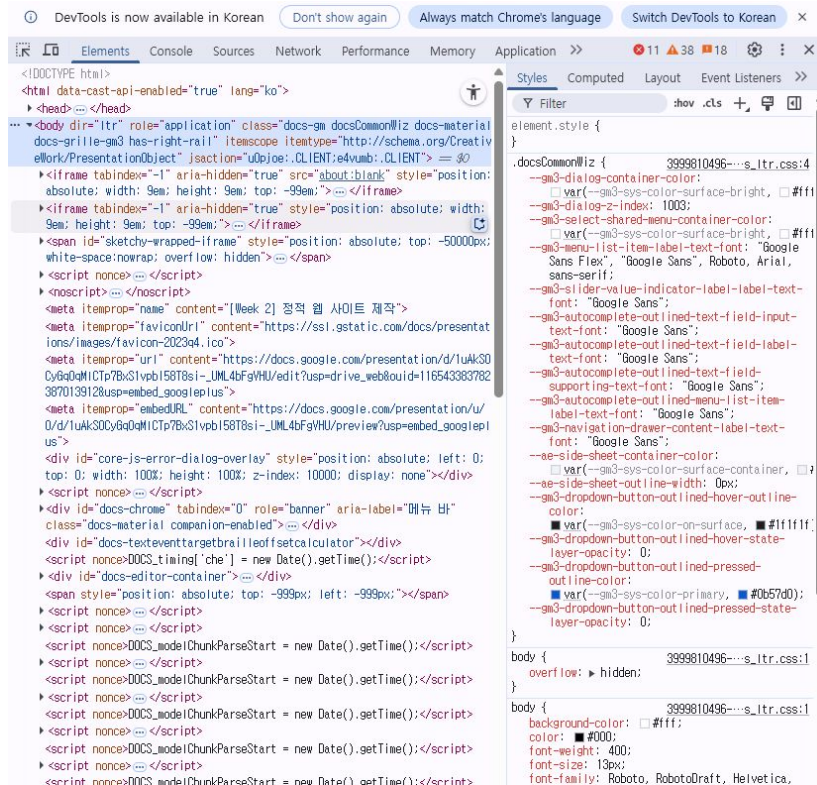
- 단축키: F12 또는 Ctrl+Shift+I (Win) / ⇧⌘I (Mac)
- 문서 요소들 확인
- 시맨틱 레이아웃
- SEO/메타 확인

반응형/뷰포트 확인

- 디바이스 톨바 도구: Ctrl+Shift+M / ⇧⌘Shift+M
- 다양한 뷰포트(모바일/태블릿)로 레이아웃 결과 확인
- 상단 Device 해상도 및 회전(rotate)도 테스트

원본 HTML & 응답 헤더 확인

- Network 패널 → Doc 타입(첫 항목) 클릭
- Headers: Content-Type: text/html; charset=utf-8 확인
- Response 탭: 서버가 보낸 원본 HTML 확인



웹 프레임워크를 활용한 개발

Svelte와 SvelteKit을 활용한 웹 프레임워크 살펴보기

웹 개발 프레임워크

웹서비스를 더 빠르고 안전하게 만들 수 있게 미리 준비된 설계도와 도구 묶음
개발과 관련된 반복 작업을 표준 방식으로 제공해 개발자의 시행착오를 줄이는 것이 목적

구성요소

- 구조: 폴더/파일 규칙, 레이어드 아키텍처(컨트롤러-서비스-모델 등)
- 러닝키트: 라우터, 미들웨어, ORM/쿼리 도구, 폼/검증, 인증, 국제화, 테스트
- 빌드/런타임: 번들링, 트랜스파일(예: TypeScript), 코드 스플리팅, SSR/SSG 지원
- 생태계: 플러그인, CLI, 베스트 프랙티스 문서

종류

- 프론트엔드(UI): React, Vue, Angular, Svelte
- 백엔드(API): Express/NestJS(Node), Django/FastAPI(Python), Spring Boot(Java), Laravel(PHP), Rails(Ruby)
- 풀스택(프론트+백엔드): Next.js, Nuxt, SvelteKit

선택 기준

- 팀 역량/언어: JS/TS - Next.js·NestJS, Python - Django/FastAPI
- 렌더링 방식: CSR/SSR/SSG/ISR 필요 여부(SEO, 초기 로드 속도 요구)
- 호스팅/배포: Vercel/Netlify(서버리스) vs 컨테이너/쿠버네티스
- 생태계: 문서/커뮤니티/플러그인 풍부함

웹 개발 프레임워크 설치

Node.js

- **Node.js**는 Chrome V8 JavaScript 엔진으로 빌드된 JavaScript 런타임 환경
- 웹 브라우저 밖에서도 JavaScript를 실행할 수 있게 해주는 프로그램

특징

1. JavaScript 런타임

- 브라우저가 없어도 컴퓨터에서 JavaScript 실행 가능
- 서버 프로그래밍, 데스크톱 앱, 명령줄 도구 개발 가능

2. 비동기 I/O (Non-blocking)

- 동시에 여러 작업 처리 가능
- 한 작업이 끝날 때까지 기다리지 않고 다른 작업 진행
- 높은 성능과 효율성

3. 단일 스레드 이벤트 루프

- 하나의 스레드로 많은 연결을 효율적으로 처리
- 메모리 사용량이 적음

4. NPM 생태계

- 세계 최대 오픈소스 라이브러리 생태계
- 100만개 이상의 패키지 사용 가능

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support

for EOL Node.js versions

CLI(명령줄 인터페이스)란?

- 키보드로 명령어를 입력해서 컴퓨터와 상호작용하는 방식
- CLI는 짧은 텍스트 명령으로 파일 관리, 개발 도구 실행, 서버 작업 등을 빠르고 정확하게 자동화

- **속도/자동화:** 반복 작업을 스크립트 한 줄로 처리
- **재현성:** 같은 명령을 실행하면 항상 같은 결과
- **원격 작업:** 클라우드/서버 관리의 표준 도구
- **개발 생태계:** Node/npm, Git, React, Express, Codex, Claude, Gemini 등 대부분 CLI로 구동

- **터미널(Terminal):** 명령을 입력하는 프로그램 (예: macOS Terminal, Windows PowerShell)
- **셸(Shell):** 명령을 해석/실행하는 엔진 (bash, zsh, PowerShell 등)
- **프롬프트(prompt):** 입력 대기 표시
- **경로(Path):** 현재 위치(작업 디렉토리). ~는 홈, . 현재, .. 상위

```
PS C:\> Get-Childitem 'MediaCenter\Music' -rec
> Measure-Object -property length -sum -min -max -ave

Count : 1387
Average : 5.91276 8956.3887
Maximum : 71.72099557
Minimum : 2.2945267
Property : Length

PS C:\> Get-WmiObject CIM_BIOSElement | select biosw*, man*, ser* | Format-List

BiosVersion : C102CPL - 6480800, Ver 1.00PARTITE)
Manufacturer : TOSHIBA
SerialNumber : N82110H

PS C:\> (cmdSearcher W)
> SELECT * FROM CIM_Job
> WHERE Priority = 1
> | @?get-CM | Format-Custom

class ManagementObject{Write-Host 2\Win32_PrintJob

    <Document> - Manual Manifesto - Public
    JobId = 6
    JobStatus =
    Owner = User
    Priority = 4
    Size = 1827888
    Name = Epson Stylus COLOR 740 ESC/T 2. 6
}

PS C:\> $rsurl = 'http://blogs.msdn.com/powershell/rss.aspx'
PS C:\> $xml = [xml](new-object System.Net.WebClient).DownloadString($rsurl)
PS C:\> $blogrss.channelelem | select title -first 3

title

RSS: What's Coming in PowerShell v2
PowerShell Presence at RSS
RSS Talk: System Center Foundation Technologies

PS C:\> $host.version, [System.Environment]::Insert(0, 'Windows PowerShell: ')
Windows PowerShell: 1.0.0.8
PS C:\>
```

웹 개발 프레임워크 설치

NPM(Node Package Manager)

- JavaScript 프로그래밍 언어를 위한 패키지 관리자
- 다른 개발자들이 만든 코드 조각(패키지)들을 쉽게 다운로드하고 관리할 수 있도록 도와주는 도구

역할

- **패키지 설치**: 필요한 라이브러리나 도구를 쉽게 설치
- **의존성 관리**: 프로젝트에 필요한 모든 패키지를 자동으로 관리
- **스크립트 실행**: 프로젝트 실행, 빌드 등의 명령어 관리
- **패키지 공유**: 자신이 만든 코드를 다른 개발자와 공유

```
npm install [패키지명]
```

```
npm --version
```

NPX(Node Package eXecutor)

- 패키지를 설치하지 않고도 실행할 수 있게 해주는 npm 패키지 실행 도구

```
# React 앱 생성  
npx create-react-app my-project
```

```
# SvelteKit 프로젝트 생성  
npx sv create my-project
```

Svelte와 SvelteKit이란?

Svelte

- Svelte는 2016년 리치 해리스(Rich Harris - <https://github.com/rich-harris>)출시한 오픈 소스 프론트엔드 웹 프레임워크
- 컴파일러를 사용하여 브라우저에서 최소한의 작업만 수행하는 간결한 컴포넌트를 작성할 수 있게 해주는 UI 프레임워크
- HTML, CSS, JavaScript를 사용하여 개발
- 2025년 9월 현재 v5.38.6
- <https://github.com/sveltejs/svelte>

Svelte

- 컴파일 타임 프레임워크
- Virtual DOM 없음
- 작은 번들 사이즈
- 빠른 런타임 성능
- 간단한 문법



SvelteKit

- Svelte 기반 풀스택 프레임워크
- 파일 기반 라우팅
- SSR/SSG 지원
- Built-in 최적화
- TypeScript 지원



주요 프론트엔드 프레임워크와 비교

Svelte의 특징

- 컴파일러 기반 → 가벼움 & 빠른 시작
- 단순한 반응성
- HTML/CSS 중심의 개발 경험
- 내장된 UX 기능
- 낮은 러닝·운영 비용

특징	React	Vue.js	Svelte
Virtual DOM	✅ 사용	✅ 사용	❌ 미사용
번들 크기	~45KB	~34KB	~10KB
학습 곡선	중간-높음	낮음-중간	낮음
성능	좋음	좋음	매우 좋음
생태계	매우 큼	큼	성장 중

파일 구조

스크립트(<script> / <script context="module">)

- 컴포넌트 로직(상태, 함수, props, 라이프사이클)을 정의
- 모듈 컨텍스트는 파일 로드 시 1회 실행.

마크업(HTML)

- 화면에 보일 구조와 콘텐츠를 작성하는 영역으로, {변수} 보간으로 데이터 출력.

스타일(<style>)

- 컴포넌트에 적용되는 CSS를 작성
- 전역 적용이 필요하면 :global(...) 사용.

svelte 파일 구조(JavaScript vs TypeScript)

```
<script>
  let name = $state('world');
  let count = $state(0);
</script>
```

```
<h1>Hello {name}!</h1>
```

```
<input bind:value={name} />
<button onclick={() => count += 1}>
  clicks: {count}
</button>
```

```
<style>
  h1 { color: #5b3df5; }
</style>
```

```
<script lang="ts">
  let name: string = $state('world');
  let count: number = $state(0);
</script>
```

```
<h1>Hello {name}!</h1>
```

```
<input bind:value={name} />
<button onclick={count += 1}>
  clicks: {count}
</button>
```

```
<style>
  h1 { color: #5b3df5; }
</style>
```

SvelteKit 프로젝트 구조

SvelteKit 프로젝트 생성

```
npx sv create my-sveltekit-app
```

```

○ my-sveltekit-app/
├── src/ # 소스 코드 (핵심)
│   ├── routes/ # 페이지와 라우팅
│   ├── lib/ # 재사용 가능한 코드
│   ├── app.html # HTML 템플릿
│   └── app.css # 전역 스타일
├── static/ # 정적 파일
├── node_modules/ # 설치된 패키지들
├── package.json # 프로젝트 설정
├── svelte.config.js # SvelteKit 설정
├── vite.config.js # Vite 번들러 설정
├── .gitignore # Git 제외 파일
└── README.md # 프로젝트 설명
  
```

```

> .svelte-kit
> e2e
> node_modules
└─ src
  └─ lib
    └─ routes
      ├── +layout.svelte
      ├── +page.svelte
      ├── page.svelte.spec.js
      ├── app.d.ts
      ├── app.html
      └── demo.spec.js
  └─ static
    ├── .gitignore
    ├── .npmrc
    ├── .prettierrc
    ├── .prettierrc
    ├── eslint.config.js
    ├── jsconfig.json
    ├── package-lock.json
    ├── package.json
    ├── playwright.config.js
    ├── README.md
    ├── svelte.config.js
    ├── vite.config.js
    └── vitest-setup-client.js
  
```

파일기반 라우팅

파일 구조가 곧 URL 구조

1. 정적 라우트 (Static Routes)

```
<!-- src/routes/about/+page.svelte -->
<h1>회사 소개</h1>
<p>우리 회사는...</p>

<!-- URL: https://mysite.com/about -->
```

2. 동적 라우트 (Dynamic Routes)

```
<!-- src/routes/blog/[id]/+page.svelte -->
<script>
  let { data } = $props();
</script>

<h1>블로그 글 #{data.id}</h1>
```

```
src/routes
├─ +layout.svelte # 앱 공통 레이아웃(헤더/푸터)
├─ +layout.js # 앱 공통 데이터 로드(선택)
├─ +page.svelte # 루트 페이지(/)
├─ dashboard/ # 정적 라우트
│   ├─ +layout.svelte # /dashboard 공통 UI(좌측 메뉴 등)
│   ├─ +layout.js # /dashboard 공통 데이터
│   └─ +page.svelte # /dashboard
│   └─ reports/ # 정적 라우트
│       └─ +page.svelte # /dashboard/reports
│       └─ [id]/ # 동적 라우트
│           └─ +page.svelte # /dashboard/reports/(동적 라우트)
```

파일기반 라우팅

3. 중첩 라우트(Nested Routes)

- 폴더 구조로 **중첩된 URL**을 만들고, 각 단계마다 공유 레이아웃(+layout.svelte)과 페이지(+page.svelte)를 구성하는 방식.
- 공통 UI**를 한 번만 선언하고 하위 페이지에서 **재사용**
- 상위 load에서 불러온 데이터를 하위에서 **parent()**로 **재사용**
- 핵심 파일**
 - +layout.svelte : 해당 폴더 이하 모든 자식 라우트의 공통 UI
 - +page.svelte : 실제 페이지 콘텐츠

```
src/routes
├── +layout.svelte # 앱 공통 레이아웃(헤더/푸터)
├── +layout.js     # 앱 공통 데이터 로드(선택)
├── +page.svelte  # 루트 페이지(/)
├── dashboard/
│   ├── +layout.svelte # /dashboard 공통 UI(좌측 메뉴 등)
│   ├── +layout.js     # /dashboard 공통 데이터
│   ├── +page.svelte  # /dashboard
│   └── reports/
│       ├── +page.svelte # /dashboard/reports
│       └── [id]/
│           └── +page.svelte # /dashboard/reports/[id] (동적 라우트)
```

변수와 상수

let 변수명 = 값, const 상수명 = 값

값 유형

\$state

- 변하는 값(상태)
- 컴포넌트에서 직접 바뀌는 값
- 배열/객체 내부 변이도 자동 추적
- 입력 값, 토글, 리스트 관리 등 UI에 바인딩되는 값

\$derived

- 계산된 값(파생값)
- 다른 상태를 읽어서 계산되는 값을 선언
- 의존성이 바뀔 때만 게으르게(lazy) 재계산
- 합계/필터링/정렬/표시용 포맷 등 순수 함수 결과

\$effect

- 부수효과(side effect)
- 반응 의존성(읽힌 상태·파생값)이 바뀔 때마다 자동 재실행
- 의존성 자동 추적: 콜백 내부에서 읽은 값들이 의존성으로 등록
- 순수 계산 금지: 계산·표현 변환은 \$derived로 분리
- 브라우저/DOM 상호작용, 타이머/루프 제어, 네트워크 요청, 로컬/세션 스토리지 동기화, 로깅/분석 트래킹

svelte 선언

```
<script>
let name = $state('world');
let on = $state(false), list = $state(['a']);
</script>

<input bind:value={name} />
<button onclick={() => on = !on}>{on ? 'ON' : 'OFF'}</button>
<button onclick={() => list.push(`item-${list.length+1}`)}>add</button>
<p>{name} / {list.length}</p>
```

```
<script>
let prices = $state([1000, 2500, 3000]);
let subtotal = $derived(prices.reduce((s,v)=>s+v,0));
let label = $derived(subtotal.toLocaleString('ko-KR')+'원');
</script>

<p>Subtotal: {subtotal} / {label}</p>
```

```
<script>
let open = $state(false);
let el;
$effect(() => { if (open && el) el.focus(); });
</script>

<button onclick={() => open = !open}>{open ? 'Close': 'Open'}</button>
{#if open}<input bind:this={el} placeholder="auto focus" />{/if}
```

조건문

스크립트(JS) 조건문

- if (조건) { ... } else if (조건) { ... } else { ... }
- 값 계산, 상태 갱신, 함수 로직

템플릿 조건 블록 (렌더 분기)

- {#if 조건} ... {:else if 조건} ... {:else} ... {/if}
- 용도: UI 렌더링 분기

svelte 조건문

```
<script>
  let score = $state(0);
  let grade = $state('C');

  // 스크립트 조건문: score가 바뀔 때 등급 계산
  $effect(() => {
    if (score >= 90)    grade = 'A';
    else if (score >= 60) grade = 'B';
    else               grade = 'C';
  });
</script>

<input type="number" min="0" max="100" bind:value={score} />

{#if grade === 'A'}
  <p>우수! (A)</p>
{:else if grade === 'B'}
  <p>보통 (B)</p>
{:else}
  <p>노력 필요 (C)</p>
{/if}
```

반복문

스크립트(JS) 반복문

- JS for, for...of, map, forEach 등 자유롭게 사용 가능.
- 결과를 UI에 쓰려면 \$state 또는 \$derived에 담아야 함.
- 데이터 가공, 계산용

템플릿 조건 블록 (렌더 분기)

- as item, i → 두 번째 인자는 인덱스
- key 지정 가능 → DOM 재사용 최적화
- 용도: UI 요소 반복 렌더링

svelte 반복문

```
<script>
  let todos = $state([
    { id: 1, text: 'Svelte 공부', done: false },
    { id: 2, text: 'Runes 연습', done: true }
  ]);

  function add() {
    todos.push({ id: Date.now(), text: '새 할 일', done: false });
  }
  function toggle(t) { t.done = !t.done; }
</script>

<button onclick={add}>추가</button>

<ul>
  {#each todos as todo (todo.id)}
    <li>
      <input type="checkbox" bind:checked={todo.done} onchange={() =>
toggle(todo)} />
      {todo.text}
    </li>
  {/each}
</ul>
```

이벤트(Event)

사용자의 행동(클릭, 입력, 키보드, 마우스 등)이 브라우저에서 발생했음을 알려주는 신호

- UI와 상호작용 구현
- on[**EventName**] = {handler}
- [**EventName**] : click, input, change, submit, keydown, mousemove...

svelte 이벤트

```
<script>
  let m = $state({ x: 0, y: 0 });

  function handleMousemove(event) {
    m.x = event.clientX;
    m.y = event.clientY;
  }
</script>

<!-- svelte-ignore a11y_no_static_element_interactions -->
<div onmousemove={handleMousemove}>
  The mouse position is {m.x} x {m.y}
</div>

<style>
  div {
    width: 100%;
    height: 100%;
  }
</style>
```

라이프사이클(LifeCycle)

컴포넌트가 생성되는 시점부터 화면에 나타나고, 사라질 때 호출되는 이벤트

- **onMount(fn)**: DOM에 마운트된 뒤 1회 실행
- **\$effect.pre(fn)**: DOM 업데이트 전 실행
- **\$effect(fn)**: DOM 업데이트 후 실행
- **onDestroy(fn)**: 컴포넌트가 파괴될 때 실행

svelte LifeCycle

```
<script>
  import { onMount, onDestroy } from 'svelte';
  let count = 0;
  let timer;

  // DOM에 붙은 뒤 1회 실행 (SSR에선 실행 안 됨)
  onMount(() => {
    timer = setInterval(() => count++, 1000);
  });

  // 업데이트 후/전에 반응
  $effect.pre(() => {
    // DOM 업데이트 "이전"에 할 작업이 있다면 여기에
    console.log('업데이트 전 count:', count);
  });

  $effect(() => {
    console.log('업데이트 후 count:', count);
  });

  onDestroy(() => {
    clearInterval(timer);
  });
</script>

<h1>{count}</h1>
```

훅(Hook)

특정 시점에 내가 코드를 끼워 넣어 실행하게 해주는 연결 지점

- 페이지 전환, 데이터 로딩, 컴포넌트 업데이트 같은 이벤트/단계를 진행할 때, 개발자가 그 순간에 실행될 콜백 함수를 등록해 동작을 확장하거나 제어
- 특정 시점에 정해진 위치에 붙여 쓰는 함수(콜백).
- 부작용 처리/제어: 로딩 스피너 표시, 네비게이션 취소, 애널리틱스 기록, 리소스 정리 등.
- 내부 동작을 바꾸지 않고, 정해진 지점만 안전하게 구현

Svelte/SvelteKit에서의 훅 예

- 컴포넌트 훅 : Lifecycle 이벤트
 - **onMount**
 - **\$effect**
 - **onDestroy**
- 네비게이션 훅 : 페이지 이동 직전/렌더 직전/이동 완료 후
 - **beforeNavigate**
 - **onNavigate**
 - **afterNavigate**
- 로드 훅: 라우팅 단계의 데이터 로딩
 - **SvelteKit의 load 함수**

svelte Navigation Hook

```
<!-- src/routes/+layout.svelte -->
<script>
  import { beforeNavigate } from '$app/navigation';

  let dirty = false; // 변경 발생 시 true로 설정

  function onInput(e) {
    dirty = true;
  }

  beforeNavigate((nav) => {
    if (!dirty) return;
    const ok = confirm('변경사항이 저장되지 않았습니다. 이동하시겠습니까?');
    if (!ok) nav.cancel();
  });
</script>

<textarea on:input={onInput} placeholder="여기에 입력해보세요..." />
```

프랍(Prop)

부모 → 자식 컴포넌트로 값을 전달하는 매개변수

`const { 변수명 } = $props();`

- 부모가 값을 바꾸면 자식 Prop도 자동 갱신
- 자식은 읽기 전용으로 사용하는 것이 원칙 (직접 변경 X)
- 여러 Prop을 한번에 받고 싶을 때 `$props()` 사용

svelte Prop

```
<!-- Child.svelte -->
<script>
  const { msg } = $props();
</script>

<p>{msg}</p>
```

```
<!-- Parent.svelte -->
<script>
  import Child from './Child.svelte';
</script>

<Child msg="Hello Prop!" />
```

양방향 바인딩

상태 ↔ UI가 동시에 동기화되는 방식.

- 입력값을 바꾸면 상태가 바뀌고, 상태를 바꾸면 입력 UI도 즉시 반영.
- 사용 : 폼 값 관리 간편화, 실시간 미리보기, 설정 슬라이더

svelte two-way binding

```
<script>
  let name = '';
  let agree = false;
  let color = 'red';
  let level = 50;
  let files;
</script>

<h2>입력 ↔ 상태</h2>

<label>
  이름:
  <input placeholder="이름 입력" bind:value={name} />
</label>
<p>미리보기: {name || '(비어있음)'}</p>
```

```
<label>
  <input type="checkbox" bind:checked={agree} />
  약관에 동의합니다
</label>
<p>동의 여부: {agree ? '예' : '아니오'}</p>

<label>
  좋아하는 색상:
  <select bind:value={color}>
    <option value="red">red</option>
    <option value="green">green</option>
    <option value="blue">blue</option>
  </select>
</label>
<p>선택: {color}</p>

<label>
  속련도: {level}
  <input type="range" min="0" max="100" bind:value={level} />
</label>

<label>
  파일 선택:
  <input type="file" multiple bind:files={files} />
</label>
{#if files}
  <p>선택한 파일 수: {files.length}</p>
{/if}
```

컴포넌트(Component)

UI를 이루는 가장 작은 재사용 단위

- 마크업 + 동작 + 상태 + 스타일을 한 덩어리로 묶어 만든 “부품”

컴포넌트가 중요한 이유

- **재사용성**: 버튼·카드·폼 필드 등 어디서든 같은 규격으로 재사용
- **가독성/유지보수**: 큰 페이지를 작은 조각으로 나눠 관리
- **테스트 용이**: 각 컴포넌트를 독립적으로 테스트
- **협업 표준화**: 팀이 공통 UI 키트를 공유

공통 핵심 개념 (프레임워크 불문)

- **Props(입력값)**: 부모 → 자식으로 내려주는 읽기 전용 데이터
- **State(내부 상태)**: 컴포넌트 안에서 바뀌는 값(클릭 수, 입력값 등)
- **이벤트/콜백**: 자식 → 부모로 알림을 올릴 때 사용
- **합성**: 컴포넌트 안에 다른 컴포넌트를 끼워 넣기
- **라이프사이클/효과**: 마운트/업데이트 시점에 실행되는 로직

svelte component

```
<!-- src/lib/Card.svelte -->
<script>
  let { title, summary, href } = $props();
</script>
```

```
<article class="card">
  <h3><a href={href}>{title}</a></h3>
  <p>{summary}</p>
</article>
```

```
<!-- src/routes/projects/+page.svelte -->
<script>
  import Card from '$lib/Card.svelte';
  let { data } = $props(); // { projects }
</script>

<h2>Projects</h2>
{#each data.projects as p}
  <Card title={p.title} summary={p.summary}
  href={` /projects/${p.slug}`} />
{:else}
  <p class="card">아직 등록된 프로젝트가 없습니다.</p>
{/each}
```

API 라우트

서버 전용 API 엔드포인트로 동작

특징

- 서버 전용 코드(비밀키, DB)를 안전하게 사용 가능
- 페이지 라우팅과 같은 폴더 구조 → 직관적인 URL 설계
- fetch로 클라이언트/서버 모두에서 쉽게 호출

위치 = 주소

- src/routes/<경로>/+server.js 파일을 만들면 그 폴더 경로가 곧 API URL
예) src/routes/api/todos/+server.js ⇒ GET /api/todos

서버 전용 실행

- +server.js는 브라우저로 변질되지 않는 서버 코드 (비밀키 · DB 접근 안전)

핸들러 = HTTP 메서드 함수 export

- 파일 안에서 export function GET/POST/PATCH/DELETE(...) { ... }처럼 메서드명과 같은 함수를 내보내면 해당 메서드 요청을 처리

요청 컨텍스트 인자

- 핸들러의 첫 번째 인자로 { request, url, params, cookies, locals, fetch } 등을 받습니다.
예) params.id(동적 세그먼트), url.searchParams(쿼리), cookies(쿠키), locals(전역 인증정보)

응답 만들기

- 표준 Response 객체 또는 SvelteKit 헬퍼로 반환
- json(data, { status }): JSON 응답
- error(status, message): 상태코드와 함께 예외 발생(자동 에러 응답)

```
src/routes
├── +layout.svelte      # 앱 공통 레이아웃(헤더/푸터)
├── +layout.js          # (선택) 클라/서버 공용 로드
├── +page.svelte        # 루트 페이지(/)
├── +page.server.js     # 루트 데이터 서버 로드
├── projects/
│   ├── +page.svelte    # /projects
│   ├── +page.js        # /projects 서버 로드(예: 위젯 요약)
│   └── api/
│       └── projects/
│           └── +server.js # /projects/api/stats (예: 위젯용 API)
```

API 라우트

api/projects/+server.js

```
// src/routes/api/projects/+server.js
import { json } from '@sveltejs/kit';

const projects = [
  { slug: 'timetable', title: 'Timetable Helper', summary: '수업/일정 정리 도우미' },
  { slug: 'gallery', title: 'Image Gallery', summary: '미니 이미지 갤러리' },
  { slug: 'memo', title: 'Memo Pad', summary: '로컬에 메모 저장' }
];

export function GET() {
  return json(projects);
}
```

projects/+page.js

```
projects/+page.js
export async function load({ fetch }) {
  const res = await fetch('/api/projects');
  return { projects: await res.json() };
}
```

projects/+page.js

```
<script>
  import Card from '$lib/Card.svelte';
  let { data } = $props(); // { projects }
</script>

<h2>Projects</h2>
{#each data.projects as p}
  <Card title={p.title} summary={p.summary}
  href={` /projects/${p.slug}`} />
{:else}
  <p class="card">아직 등록된 프로젝트가 없습니다.</p>
{/each}
```

Playground

<https://svelte.dev/playground/>

The screenshot displays the Svelte Playground web application. At the top, the Svelte logo and navigation links (Docs, Tutorial, Playground, Blog) are visible. A search bar and social media icons are on the right. Below the navigation, the editor title is 'Hello world (edited)'. The code editor shows the following Svelte code:

```
1 <script>
2   let name = $state('world');
3   let count = $state(0);
4 </script>
5
6 <h1>Hello {name}!</h1>
7
8 <input bind:value={name} />
9 <button onclick={() => count += 1}>
10   clicks: {count}
11 </button>
12
```

Below the code editor, there are tabs for 'Result', 'JS output', 'CSS output', and 'AST output'. The 'Result' tab is active, showing a live preview of the code. The preview displays 'Hello world!' in a large font, followed by an input field containing 'world' and a button labeled 'clicks: 0'.

웹 서비스의 배포 자동화

CI/CD

CI (Continuous Integration)

- 지속적인 통합
- 새로운 코드 변경 사항이 정기적으로 빌드 및 테스트 되어 공유 저장소에 통합하는 것

핵심 목표

- 버그를 신속하게 찾아 해결
- 소프트웨어의 품질 개선
- 새로운 업데이트의 검증 및 릴리즈의 시간 단축

CD (Continuous Delivery & Deployment)

- **Continuous Delivery** : 개발자의 변경 사항이 저장소에 Release
- **Continuous Deployment** : 프러덕션 환경까지 자동으로 Deploy

핵심 목표

- 개발팀과 비즈니스팀 간의 커뮤니케이션 문제를 해결
- 배포에 이르기 까지 노력을 최소화

DevOps

- 개발(Development)과 운영(Operations)을 연결해, 코드 작성부터 배포·운영까지 전 과정을 자동화·협업·최적화하는 역할

Vercel

Vercel 소개 <https://vercel.com/>

- 클라우드 컴퓨팅 서비스이자 미국 샌프란시스코에 위치한 스타트업 기업이자 클라우드 서비스
- React기반 SSR 프레임워크인 Next.js를 개발.
- Svelte - 2021년 11월 스벨트의 개발자인 리치 해리스를 풀타임으로 고용

Vercel 특징

- 프론트엔드 클라우드 플랫폼
- 정적(Static)사이트와 고수준의 서버리스 기능 제공
- 자동 CI/CD 파이프라인
- 글로벌 CDN 제공
- 무료 SSL 인증서
- 실시간 미리보기
- 성능 분석 도구



Vercel 시작하기

Vercel 계정 생성

- vercel.com 접속
- GitHub 계정으로 가입 (권장)

GitHub 저장소 연결

- Vercel은 GitHub 저장소의 변경사항을 자동으로 감지하고 재배포.

1. "Import Git Repository" 선택
2. GitHub 계정 연동 및 권한 부여
3. 배포할 저장소 선택
4. 프레임워크 자동 감지 (SvelteKit)
5. 환경 변수 설정 (필요시)
6. "Deploy" 클릭

실습: SvelteKit을 이용한 웹사이트 제작과 Vercel 배포

실습 개요

SvelteKit을 이용한 웹사이트 제작과 Vercel 배포

학습 목표

- Node.js와 npm 환경 구축
- SvelteKit 프레임워크로 정적 사이트 제작
- Git/GitHub를 활용한 버전 관리
- Vercel을 통한 자동 배포 파이프라인 구축

요구사항

- GitHub 저장소에 프로젝트 버전 관리가 바르게 되고 있는가?
- Vercel에 GitHub 저장소가 연결되어 자동 배포가 이루어 지고 있는가?
- Vercel 배포한 웹서비스에서 제시한 프로젝트 코드가 모두 바르게 동작하는가? (Home, About, Projects)

참고 : <https://github.com/ajou-hyunseok-oh/pwd-week2>

과제 마감일

- 9월 14일 11시 59분 까지
- GitHub 저장소 URL
- (예: <https://github.com/username/pwd-week2>)
- Vercel App URL
- (예: [https://pwd-week2-\[random\].vercel.app](https://pwd-week2-[random].vercel.app))

Q&A